

6.2 Audio Encoders and Speech Models

Whisper transcribes (GPT-style decoder, no CTC). CLAP retrieves. Same spectrogram, different head.

These notes match the lecture slides. Use **(slides)** on the course hub for the deck.

Two audio systems you will cite: **Whisper** transcribes (encoder–decoder), **CLAP** retrieves (two towers, like CLIP). Same spectrogram tokens, different heads. Whisper: log-mel in, two convs, a transformer encoder, a GPT-style decoder, 30-second chunks, no CTC.

First time this method appears. **Whisper** transcribes speech. **CLAP** retrieves audio against text. Same spectrogram, different head.

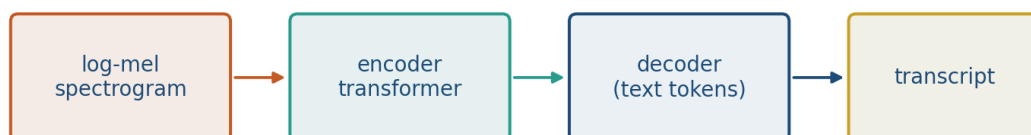
What. Whisper: encoder (audio tokens) + **GPT-style decoder** that writes text (no CTC required in this course). CLAP: two towers like CLIP, audio instead of pixels. **Why.** ASR needs a decoder. Retrieval needs a score. Do not use Whisper as a search index or CLAP as a transcriber. **Architecture.** Whisper: spectrogram → conv/embed → encoder blocks → decoder with cross-attention. CLAP: audio encoder f , text encoder g , cosine. **How.** Whisper: teacher-forced captions of speech, then generate. CLAP: InfoNCE on audio–text pairs (same loss family as 4.3). **Formula.** Whisper LM: $\prod_t p(y_t | y_{<t}, \text{audio})$. CLAP: $\cos(f(a), g(t))$. **Tradeoffs.** + Whisper is a generator; CLAP is cheap retrieval. – Whisper is not a general audio tagger; CLAP does not decode words; paired audio–text is scarce (note 6.3).

1. Whisper: listen, then write

Radford et al. (2022). The encoder is a transformer on log-mel spectrogram patches (note 6.1). Encoder input is the log-mel plus **two convolutional layers**, then positional embeddings, then standard transformer blocks. The decoder is causal and **cross-attends** to the encoder, like translation in note 3.3. It is a **GPT-2 style** decoder. There is **no CTC loss**.

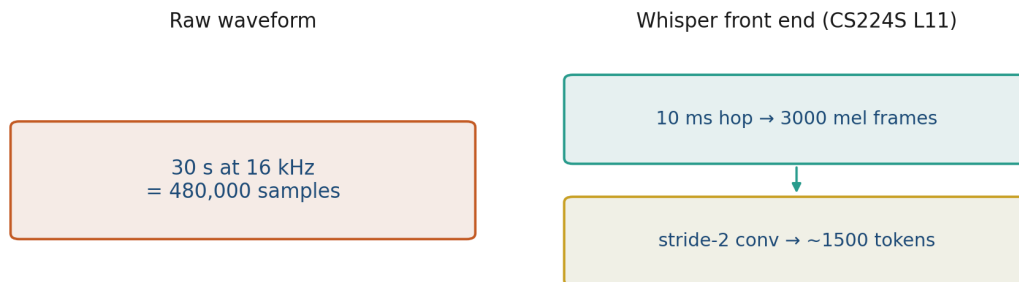
The output is text, plus special tokens for language, timestamps, and task (transcribe versus translate).

Whisper-style: audio encoder, text decoder (Week 3 block, new input)



Pretraining is weakly supervised: a huge pile of audio paired with transcripts from the web, filtered rather than hand-labeled (on the order of 680,000 hours). Multilingual is the point. You do not train this from scratch in the lab; you would load a checkpoint. The architecture is what you need to draw. Default front end: 25 ms windows, 10 ms hop, 80 mel bins, **30-second** chunks. Thirty seconds at a 10 ms hop is 3000 frames before conv downsampling. The decoder never sees raw samples; it sees encoder states.

The spectrogram exists so the encoder is not a 480k-token transformer.



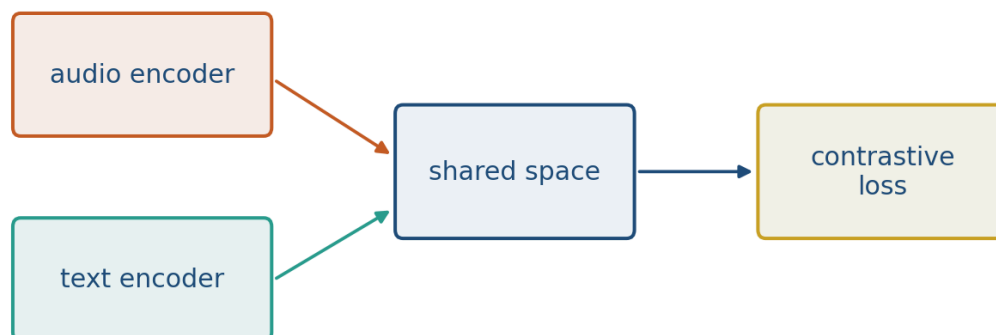
Draw the two arrows: encoder self-attention over time–frequency tokens, decoder causal self-attention plus **cross-attention** into those states. That is the same picture as translation (note 3.3), with a spectrogram instead of a source sentence.

Long-form audio: Whisper only sees 30 s at a time. Timestamp tokens tell it when to shift the window. That is a product detail, not a new architecture.

2. CLAP: CLIP, but the image is sound

CLAP (Contrastive Language–Audio Pretraining) is coordinated, not joint. An audio encoder (CNN or transformer on the spectrogram) and a text encoder emit vectors in one space. InfoNCE on audio–caption pairs. After training you keep both towers.

CLAP is CLIP for audio: two towers, cosine, paired captions



Zero-shot tagging: encode "a dog barking", rank clips by cosine. Retrieval both ways. It is not a transcriber. If you need words, use Whisper (or a CTC ASR). If you need "find the clip that matches this sentence," use CLAP.

The text tower can be bidirectional (BERT-style) because you encode a **whole** caption as one vector. Whisper's decoder is causal because it **emits** tokens left to right.

InfoNCE on a batch of B pairs is the CLIP loss from note **4.3**: one audio vector per clip, one text vector per caption, a $B \times B$ cosine matrix, positives on the diagonal. After training you throw away the matrix and keep the two encoders.

3. Which one in a project

Need	Model
Transcript, subtitles, timestamps	Whisper
Search / zero-shot tags / audio-text retrieval	CLAP
Generate a caption in free text	a decoder with an audio encoder (Whisper-like, or a BLIP-style audio LM)

Fusing audio with vision or text is next (note **6.3**). Do not concatenate a Whisper hidden state into CLIP and call it a paper; say how you aligned the rates and the losses.

4. Teaching this note

About **30–35 minutes** at the board: draw the Whisper encoder-decoder (cross-attention arrows, 30 s chunk, special tokens), then the CLAP two-tower picture, then fill the table with a project prompt from the room. Play the Whisper video **in full** (~5–12 min). This is not Lab 6; the lab stays on a sine STFT and toy fusion.

Board order: (1) 30 s $\rightarrow$ 3000 frames, (2) decoder special tokens and "no CTC," (3) CLAP cosine rank, (4) "which model for this app?" You should leave the hour able to pick Whisper vs CLAP without hedging.

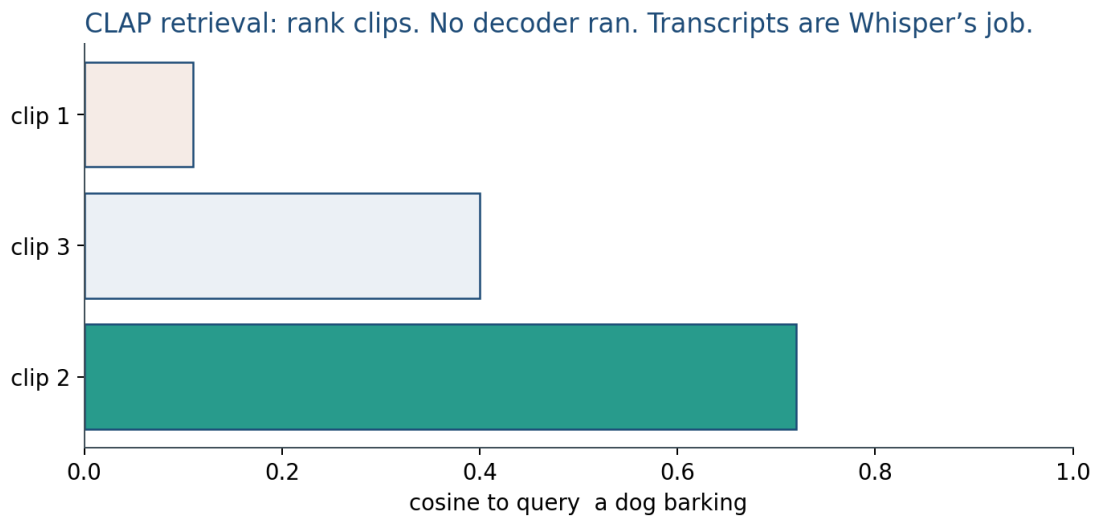
If someone says "both take spectrograms so they are the same," stop and point at the head: causal decoder vs a cosine.

5. Worked example

Whisper chunk: 30 s, hop 10 ms $\Rightarrow 30 / 0.010 = 3000$ mel frames, 80 bands. After a stride-2 conv (as in the paper's stem, two convs), you are on the order of 1500 encoder time-steps, not $30 \text{ s} \times 16 \text{ kHz} = 480,000$ waveform samples. That is why the spectrogram exists.

Decoder: special token `<|transcribe|>` then English text. The loss is next-token on those text tokens, **conditioned** on the encoder. Timestamp tokens are just more vocabulary; they are not a separate model.

CLAP retrieval: three clips with audio embeddings a_1, a_2, a_3 and a query text t . Cosines 0.11, 0.72, 0.40. Rank is clip 2, then 3, then 1. No decoder ran. If the user wanted a transcript of clip 2, you would call Whisper on the waveform, not on a_2 .



6. Where students get stuck

- Calling CLAP a “speech-to-text model” because both take spectrograms. The head is the task.
- Forgetting Whisper’s decoder is **causal** and CLAP’s text tower need not be.
- Stuffing Whisper encoder states into a CLIP image slot without resampling time. Rates (10 ms frames vs patch tokens) have to be named.
- Assuming Whisper still uses CTC. Whisper’s decoder is GPT-2 style; there is no CTC.

7. Video

[What's AI — OpenAI's Whisper Model Explained](#). Play **0:00–end**. Pause on the encoder–decoder sketch and on the 30-second chunk. CLAP is the board comparison; this clip is Whisper only.

8. Practice

1. Whisper’s decoder is causal. Why can CLAP’s text tower be bidirectional like BERT?
2. You have a gallery of podcast clips and a typed query. Whisper or CLAP? Why?
3. Name one special token Whisper uses that CLIP never needed.

4. A 10 s clip, 10 ms hop, 80 mel bins, no extra pooling. How many time frames hit the encoder? If you then cut non-overlapping 2-frame patches along time only, how many patch tokens?
5. Two captions score 0.9 and 0.1 cosine against one clip. You want words that were actually spoken. Which model, and what does the 0.9 number **not** tell you?